Compositional Inference: Localized KV-Cache Repair and Specialist Routing

Author: Abhishek Pant
Affiliation: Independent Researcher
Education: MCA (AI/ML specialisation), 2025 graduate
Profile: LinkedIn
Date: October 2026
Repository: github.com/Rusty-user365/AGI
Artifacts: `experiments/path_a_loop.py`, `results/path_a_loop_h5.json`, `findings/2026-10-28-path-a-ensemble-kv-slicing.md`

Abstract

We study a compositional inference framework that combines frozen language-model backbones, learned error localization, bounded interventions, and task-specific low-rank adapters. The intended benefit is modular repair and specialist selection without updating the base weights at inference. This is a proposed alternative to retraining a monolithic system for every specialized behavior; we do not establish a solution to catastrophic forgetting or a replacement for jointly trained mixtures of experts. On five-step arithmetic with Qwen3-0.6B-Base, a frozen rewind-and-token-exclusion loop recovers 73/331 already-broken chains versus 46/331 for full rewrite on a blind vault. That comparison is repair conditional on a known failure: every evaluated chain is complete and already wrong, selected using ground-truth first-error labels, so neither detection false positives nor damage to correct chains is measured. The counts are candidate-level, and candidates drawn from one task share that task, so the paired tests treat more units as independent than the design supports. Timing claims are withdrawn: the saved cache measurements crop a prefix cache to its own length and stop before the repaired suffix is generated, so they are not equal-work speedups. Separately, on the chat-capable Qwen3-0.6B, an exploratory correctness router trained on 200 SecEval questions achieves 57/80 on CyberMetric-80 versus 54/80 for both the base and a cyber adapter. A live implementation reproduces all 80 base answers and routing decisions and all nine fresh-cache specialist outputs; three switches help and none harm on this sample, an exact paired result of $p = 0.25$. These are separate experiments with different backbones and tasks, not a demonstrated end-to-end multi-domain DAG system. The public benchmark was previously examined, possible dataset contamination is unknown, and the routing gain is preliminary.

Contributions and results at a glance

Across the arithmetic and exploratory adapter experiments, we report:

1. **No Demonstrated Advantage for Heuristic Baselines:** A sequence-level pooled observer did not select correct trajectories better than the model's own raw token logprob ($\mathrm{AUC} = 0.682$ $[0.573, 0.782]$; selection $+0.004$, $12\mathrm{W}/11\mathrm{L}$, $p = 1.000$). Natural-language hindsight constraints matched a neutral placebo (27.3% vs 27.3%, $15\mathrm{W}/15\mathrm{L}$, $p = 1.000$). Neither comparison is an equivalence test and neither rules out the method class; both are null results under these pooling, prompt and training conditions.
2. **Step Localization, and the Threshold That Over-Triggers:** An independently trained per-step MLP, fixed during evaluation, over SVD-32 hidden states (historical code name `LinearStep`) achieves 0.923 recall on early divergence (Step 1) but collapses to 0.188 on tail steps (Step 4). A cross-step Transformer encoder supplies complementary tail sensitivity (0.500 Step-4 recall in-sample). The two

arms do **not** share an input representation — the MLP receives SVD-32 projections, the Transformer full-width standardized features — and they differ in parameter count and positional handling as well, so this comparison does not isolate attention. An asymmetric Two-Tier rule ($P_{trf}(4) \geq 0.35$) lifts Step-4 recall to 0.812 and Top-1 to 0.718 on the tuning holdout. Out of sample the rule does not collapse — it over-fires. On a frozen blind vault ($N = 331$) it calls Step 4 on 68 rows where only 33 are Step 4, which gives it the **highest Step-4 recall of any arm (0.667 vs 0.545 for the Transformer) at the lowest Step-4 precision (0.324 vs 0.400)**, while the original fit had lower aggregate Top-1 (0.625 vs 0.677). The smooth softmax ensemble has the highest reported aggregate vault Top-1 (0.680 Top-1). The Step-4 counts and recall above come from a later re-fit, whereas Top-1 comes from the original fit; they must not be interpreted as one saved prediction set (§4.3).

3. **Loop Recovery Conditional on Known Failure:** Driving a rewind loop with localization and rank-2 first-digit exclusion reaches 29.1% on the $N = 110$ tuning holdout. On the blind vault the frozen loop recovers 22.1% ($73/331$) against 13.9% ($46/331$) for full rewrites ($43\text{W}/16\text{L}$, $p = 5.84 \times 10^{-4}$), a $1.59\times$ count ratio. Every evaluated chain is already broken and complete, and the broken set is selected using ground-truth `first_error`, so these are repair rates given failure, not detection rates or end-to-end accuracy. The counts are candidate-level; the paired tests ignore clustering within tasks. The $2.29\times$ figure sometimes quoted pairs the same loop against a different draw of the stochastic rewrite baseline and is not the headline.

4. **Cache Timings Are Not Equal-Work Speedups (Withdrawn):** The saved cache block builds a fresh prefix cache, calls `crop` with that cache's existing length, stops the timer before the repaired suffix is generated, and excludes localization and the gate. It therefore measures a prefill fragment, not cached repair generation. The recorded values — $581.1 \rightarrow 119.7$ tokens and $4{,}498.4 \rightarrow 173.7$ ms — are retained as historical prefix-prefill records with a $25.9\times$ ratio that must not be read as a generation or loop speedup. Token counts in every arm are formulas (prompt length $+2$ per step), not forward-pass instrumentation. A replacement benchmark must build a cache over a full failed trajectory, truncate it to a boundary strictly inside that trajectory, generate and verify the repaired suffix, and price localization and gating inside the same timer. Separately, a CPU numerical check confirms that `DynamicCache.crop` continuation matches fresh recomputation to 1.86×10^{-5} maximum logit difference with matching argmax; that supports coherence of the operation, not a speedup.

5. **Out-of-Sample Vault Replication:** On an independent holdout vault ($N = 331$ unseen broken chains, seed 20261028), frozen CGR recovers 22.1%, beating full rewrites (13.9%, $43\text{W}/16\text{L}$, $p = 5.84 \times 10^{-4}$) and a random-step permutation control (6.9%, $57\text{W}/7\text{L}$, $p = 7.64 \times 10^{-11}$). These p-values are candidate-level and do not account for multiple candidates per task; the vault artifact stores aggregate wins and losses, not row-level outcomes, so a task-clustered test cannot be reconstructed from it. A historical narrative reports a separate slicing comparison; the independently saved CPU check is described in §4.2. The previously reported $1.70\times$ loop speedup and $27.2\times$ generation-slice speedup are withdrawn for the reasons given in item 4.

6. **Live specialist routing:** On a previously inspected 80-question public cybersecurity benchmark, the frozen SecEval-trained binary router scores 57/80 versus 54/80 for the base and the cyber adapter; a per-question oracle scores 61/80. The live loop regenerates all base answers, reproduces all nine selected cyber outputs after full-prompt prefill from an empty cache, and observes three useful and zero harmful switches in this sample; the exact paired test on 3 versus 0 gives $p = 0.25$, so this is an exploratory observation rather than established superiority. Neither the base nor adapter weights are updated during inference, although the router heads were supervised-trained. This validates a narrow implementation, not universal safety or general multi-step compositional recovery.

Introduction

A long generated solution may contain many steps that are useful even after a later step fails. Consider an agent constructing a program in 20 steps: a defect discovered at step 20 might originate in step 6, while earlier validated work and some independent artifacts remain useful. A full rewrite discards this work; repeated attempts by the same model may also reproduce the same error. Other systems can checkpoint or edit their work, so full regeneration is a comparison baseline, not an unavoidable property of all agents.

This paper investigates whether an inference controller can keep the verified prefix, locate a suspected failure, and repair only the necessary portion. A dependency graph records which artifacts depend on the changed step: descendants require revalidation or regeneration, while independent artifacts can be preserved. The attention cache has a stricter constraint: even preserved artifacts that occur after a changed prefix may need their tokens replayed, because their cached states were computed in the old context. The controller therefore records the pre-error token boundary, truncates the cache to that boundary, and rebuilds later state rather than assuming artifact independence implies cache independence.

A separately recorded CPU float32 diagnostic checks physical cache truncation on one fixed prompt: recomputing its suffix after cropping matches the fresh full-prefill next-token argmax, with maximum logit difference 1.86e-5. This supports that tested operation, not a general equivalence theorem or latency advantage. Historical arithmetic speedup interpretations are withdrawn (§4.2). Similarly, the tested textual hindsight constraint showed no advantage over its neutral placebo; this does not establish that language-based self-correction is generally ineffective.

The architecture also explores a second route to a different repair: a separately trained specialist adapter can generate a candidate after a verifier rejects a result. Accepted specialist tokens can be replayed under the base model through a bridge before continuation. We report a natural base-failure/coder-pass example and a separate four-node coder/cyber card fixture, including its failed first coder attempt and verifier-feedback retry. We also report a live binary cyber-router experiment. These results test components and small integrations; they do not yet establish a complete 20-step delayed-failure graph, a universally reliable error detector, or safe deployment across domains.

The motivation is broader than any one model size: if failures can be identified and useful state preserved, localized repair may avoid unnecessary recomputation. Whether the same approach helps larger models, frontier agents or different hardware is an open empirical question. We do not assume that a larger model always repairs an error, or that adapters and caches can be interchanged between different base models.

Relation to prior work

Memory-based feedback without weight updates is not new: Shinn et al., Reflexion maintain reflective text across trials. Our manually authored rule controls do not establish an advantage over that approach. Huang et al. study intrinsic self-correction without external feedback; that setting differs from repair using deterministic verifiers or supervised localization. Sheng et al., S-LoRA address serving many adapters and managing adapter/cache memory. We do not claim adapter serving or memory reuse as new primitives. This paper investigates a particular combination of conditional localization, intervention and specialist routing; it has not established superiority to these systems or a unified end-to-end implementation.

---

1. Preliminaries & Rigorous Negative Controls

Autonomous error recovery in language models requires answering three nested questions:

- *Detection:* Does the model's internal representation signal when a mistake occurred?
- *Localization:* Can an observer determine *which* discrete step was the first point of divergence?

- *Intervention:* Can the generation be steered away from the error without re-running sound prefix computation?

Before building the step-localizer, we tested three common hypotheses in the literature. None showed an advantage over its control in these experiments; none is refuted by them.

1.1 Sequence-Level Observer Ranking Null

A persistent conjecture is that an auxiliary sequence-level observer head trained on pooled sequence representations can rerank sampled model outputs more effectively than token logprobs.

We evaluated a 2-layer sequence-level MLP head across $n = 500$ multi-step arithmetic tasks ($1,096$ wrong-but-well-formed holdout candidates). While the head achieved above-chance discrimination ($\mathrm{AUC} = 0.682\ [0.573, 0.782]$ vs permutation control 0.496), it did not outperform the free baseline as a generation selector:

$$\text{Head Selection Top-1: } 0.560 \quad \text{vs} \quad \text{Max-Logprob: } 0.556 \quad (+0.004,\ 12\mathrm{W}/11\mathrm{L},\ p = 1.000)$$

The observer's $+0.004$ selection advantage over the model's own token logprobs is within noise ($12\mathrm{W}/11\mathrm{L}$, $p = 1.000$). This is a failure to demonstrate an advantage, not a demonstration of equivalence. One plausible explanation, not established here, is that sequence pooling averages away position-specific evidence: an early catastrophic arithmetic failure and a subtle late rounding error can produce similar mean activation drift.

1.2 Sub-1B Symbolic State-Tracking Floor

We probed whether small autoregressive models ($0.6\mathrm{B}$–$1.5\mathrm{B}$) maintain internal symbolic state tracking on multi-entity physical state tasks (e.g., tracking colored balls across boxes). On single 1-ball displacement tasks:

- `Qwen3-0.6B-Base`: 22.5% accuracy (2 boxes), collapsing to 2.5% on 3–4 boxes.
- `Qwen2.5-1.5B-Instruct`: 2.5% raw greedy, 15.0% in chat format.

When Step 1 fails in 77.5% of instances, the ground-truth first-error index is degenerately $t = 0$, rendering step localization ill-defined. To establish a valid instrument, we designed `arith_steps`: sequential assignment chains with strict syntax compliance (`x = 5`, `y = x + 3`, ...), where Step 0 is virtually error-free (0.991) and error divergence spreads across later steps.

1.3 Capacity Control & In-Distribution Steering Nulls

In early modular plastic memory experiments (CGR/FiLM architecture), the apparent gating gain was confounded by capacity. Against a LoRA-only checkpoint it looked decisive — $+0.135\ [0.060, 0.210]$, significant — but that control trained $442{,}368$ parameters against CGR's $8{,}149{,}661$, so the comparison confounded conditioning with a $19\times$ capacity gap (`findings/2026-10-25-steering-capacity-control.md`). Raising the baseline to a matched $22{,}324{,}637$ trainable parameters, with the same corpus, seed and schedule, moved it from 0.245 to 0.350 while CGR moved from 0.380 to 0.385. The matched paired difference is $+0.035\ [-0.045, +0.115]$ — **not significant** — and the one significant difference runs the wrong way: the steered arm damages edit quality by $-0.156\ [-0.240, -0.065]$ while restoring the thesis no more reliably than the control.

Erratum (2026-10-29): an earlier draft reported this control as "$+0.105$ vanished when matched against parameter-equivalent LoRA adapters ($8.15\mathrm{M}$ vs $442\mathrm{K}$)". No artifact contains that pairing: `8.15M` vs `442K` is the *unmatched* comparison, which produced $+0.135$, not $+0.105$, and $+0.105$ appears nowhere in `results/`. The matched control is $22.3\mathrm{M}$ vs $22.3\mathrm{M}$. See Errata at the end.

Summary: Heuristic rerankers, ungrounded prompt tweaks and parameter-unmatched gates showed no advantage over their controls in these experiments. That is a null result under these conditions, not a proof that the method classes cannot work. Progress in this project required explicit step-level credit assignment grounded in physical key-value states.

---

2. Un-Pooled Step Localization & The Two-Tier Architecture

To localize errors, we construct an un-pooled representation $\mathbf{H} \in \mathbb{R}^{H \times d}$, where row t contains the mean hidden state across the tokens of the emitted value at step t ($H = 5, d = 1024$).

2.1 Why Confidence and Heuristics Fail

We evaluated four non-learned baselines on $N = 110$ held-out broken chains:

Baseline Arm	Mechanism	Top-1 Accuracy	Balanced Accuracy	Description
`chance`	Uniform $1/H$	0.200	0.200	Theoretical random guess
`modal_prior`	Always predict Step 1	0.355	0.200	Empirical majority step
`argmin_logprob`	$\arg\min_t \log P(v_t)$	0.400	0.293	Predicts Step 4 in 93/110 cases
`poslogprob`	Position bias + slope	0.445	0.347	10 params; non-modal accuracy collapses to 0.141

In this sample, `argmin_logprob` selects the final step on 93/110 chains. This shows a strong position-associated bias for this heuristic, not that token confidence is universally uninformative or that positional drift has been isolated as its cause.

2.2 Per-Step MLP vs. Transformer Localizers: Complementary Defect Profiles

We trained two observer families on frozen hidden states. They do **not** share an input representation, which bounds how the comparison can be read:

1. **Per-step MLP (historical code name `LinearStep`):** an independent two-layer MLP applied to each step, `Linear(d, 64) → GELU → Dropout(0.1) → Linear(64, 1)`, with SVD-32 projections of the step features and no cross-step attention.
2. **Transformer Encoder:** a 1-layer bidirectional self-attention encoder over the H step embeddings, fed full-width standardized features.

The width difference is not incidental. `path_a_loop.py:91-92` and `eval_vault.py:76-77` pass the SVD projection (`proj`) to the MLP arm and `None` to the Transformer arm, so the Transformer sees the full feature width while the MLP sees 32 dimensions. The two arms therefore differ in input dimension, parameter count, positional handling and attention simultaneously. The architectural description follows the loop code, whose `localizer_eval` block records Table 1 diagnostics. Historical class names are retained in artifact keys for traceability.

Both families reach the same aggregate Top-1 accuracy (0.618) on the tuning holdout, but their per-step error profiles differ:

```
Step 1 Divergence: [MLP Recall: 0.923]  >>>  [Transformer Recall: 0.769]  (MLP +0.154)
Step 4 Divergence: [MLP Recall: 0.188]  <<<  [Transformer Recall: 0.500]  (Trf +0.312)
```

The per-step MLP defaults to Step 1 in 59% of instances and is weak on late-chain errors (Step-4 recall 0.188). One candidate explanation for the Transformer's tail-step behaviour is that bidirectional attention lets a late step be compared against earlier ones. That is a hypothesis, not an isolated effect: the comparison confounds attention with feature width, parameter count and positional encoding, and §2.4 states what would be needed to separate them.

2.3 The Threshold Over-Fires Out-of-Sample

Because the error distributions are complementary, a threshold on the transformer's step-4 posterior can lift tail recall without training a large unified model. It does so on the split the threshold is chosen on, and not out of sample — which is the point of this section:

$$\hat{t} = \begin{cases} 4, & \text{if } P_{trf}(t=4) \geq 0.35 \\ \arg\max_t P_{lin}(t), & \text{otherwise} \end{cases}$$

Table 1: Localizer Diagnostic Performance on the Tuning Holdout ($N = 110$, 5 Seeds) — In-Sample for τ

Architecture	Top-1 Accuracy	Balanced Accuracy	Step 1 Recall ($n = 39$)	Step 2 Recall ($n = 23$)	Step 3 Recall ($n = 13$)	Step 4 Recall ($n = 32$)
Per-step MLP (`LinearStep`, SVD_{32})	0.618	0.515	0.923	0.696	0.769	0.188
Transformer Majority	0.618	0.505	0.769	0.565	0.692	0.500
Ensemble (Softmax Mean)	0.691	0.571	0.897	0.783	0.769	0.406
Two-Tier Localizer	0.718	0.559	0.846	0.522	0.615	0.812

The Two-Tier localizer achieves a $4.3\times$ **gain in Step-4 recall** ($0.188 \rightarrow 0.812$), lifting aggregate Top-1 localization accuracy from 0.618 to 0.718 ($+10.0\%$ absolute) — **on the split used to choose** $\tau = 0.35$.

Parameter counts are not matched. From the module definitions (`experiments/localizer.py:124-164`) at the loop's own settings (width 64, one encoder layer, four heads, $H = 5$), the per-step MLP has $2{,}177$ parameters at $d = 32$, while the Transformer has $99{,}585$ at $d = 1{,}024$, of which $65{,}600$ are in its input projection alone. The Transformer therefore has roughly $46\times$ the parameters of the MLP it is compared against. These counts are computed from the constructors, not read from a saved record.

That last qualification is load-bearing. On the frozen blind vault ($N = 331$, Table 5) the same rule scores 0.625 Top-1, below the Transformer alone (0.677) and below the softmax ensemble (0.680). It is not failing to localize tail steps, though: it calls Step 4 on 68 of 331 rows when only 33 are Step 4, which lifts Step-4 recall to 0.667 (Transformer 0.545, ensemble 0.485) and drops Step-4 precision to 0.324 (0.400, 0.471). A single scalar threshold on one class posterior is a fitted recall lever, and out of sample it over-fires. The ensemble uses no class-specific cutoff and retained similar aggregate accuracy on this vault; the Two-Tier rule is reported here as the tuning-set maximum, not as the recommended model.

2.4 Attention or Confounded Capacity? An Open Question

A skeptical reviewer should ask the question that ended the Phase 1 auxiliary-head claim: *Is the Transformer localizer winning because of cross-step attention, or because it has more trainable parameters and a wider input?*

The present experiments cannot answer it, and we no longer claim they do. The two arms differ simultaneously in input dimension (SVD_{32} vs full width), parameter count ($2{,}177$ vs $99{,}585$ at the loop's settings; §2.2), positional handling, and the presence of cross-step attention. No width-, parameter- or input-matched per-step MLP control, no position-only control, and no attention-disabled counterpart was run for this comparison. The FiLM capacity control in §1.3 matched a different architecture on a different task and does not control this one. What the data support is a descriptive asymmetry, not a causal decomposition:

1. **Per-step asymmetry (descriptive).** The per-step MLP reaches 0.923 Step-1 recall, while the Transformer reaches 0.500 Step-4 recall against the MLP's 0.188 on the tuning holdout, and 0.545 against 0.394 on the vault re-fit. The two error profiles are complementary in this configuration.
2. **A hypothesis, not a mechanism.** One reading is that an early error perturbs a single step's state enough for an independent per-step classifier, while a late error is separable only in relation to earlier steps. Residual-stream drift, positional encoding, attention mixing and the wider input are all candidate contributors. Per-step error asymmetry cannot distinguish them, and no experiment here does.
3. **What would count as evidence.** A width-, parameter- and input-matched per-step MLP; a position-only variant; a Transformer with attention disabled but the same input projection; and the same comparison on both the tuning split and the vault. Until those exist, the honest statement is that cross-step attention is a plausible contributor and its contribution is unmeasured.
4. **The Two-Tier synthesis is operational, not mechanistic.** Tier 1 runs the per-step MLP on early steps; Tier 2 overrides with the Transformer when $P_{trf}(4) \geq 0.35$. That combination performs well on the split where τ was chosen and over-fires out of sample (§2.3). It is a fitted recall lever built from two differently configured arms, not a validated theory of relational inductive bias.

---

3. Mechanism of Repair: Hindsight Placebo vs. Token Exclusion

Localization specifies *where* to intervene, but not *how*. We evaluated seven intervention mechanisms under an **Oracle Localizer** (intervening at the true divergence step t^*) across all 110 broken holdout chains.

Scope condition. The 110 chains are complete and already wrong, selected by the dataset loader using ground-truth `first_error` (`experiments/localizer.py:68-85`). Malformed or incomplete chains are excluded, and chains that were generated correctly are absent. Every rate in this section is therefore *recovery given a known failure at a known step*, not detection or end-to-end accuracy, and not a measure of how often the mechanism damages a correct chain.

Table 2: Oracle Intervention Taxonomy ($N = 110$ Broken Chains, Qwen3-0.6B-Base)

Arm	Perturbation Mechanism	Recovered / N	Rate	95% Wilson CI	Re-emitted Bad Value	Paired vs Rewrite
`full_rewrite`	Regenerate all 5 steps ($T = 0.8$)	20 / 110	0.182	$[0.121, 0.264]$	—	—
`rewind_greedy`	Greedy replay from t^*	17 / 110	0.155	$[0.099, 0.234]$	0.618	$6\text{W}/9\text{L}, p = 0.61$
`resample`	Sample step t^* ($T = 0.8$), greedy after	22 / 110	0.200	$[0.136, 0.284]$	0.445	$15\text{W}/13\text{L}, p = 0.85$
`hindsight`	Prompt: `# constraint: x is not <bad>`	30 / 110	0.273	$[0.198, 0.363]$	0.136	$23\text{W}/13\text{L}, p = 0.13$

Arm	Perturbation Mechanism	Recovered / N	Rate	95% Wilson CI	Re-emitted Bad Value	Paired vs Rewrite
`hindsight_neutral`	Prompt: `# constraint: none` (Placebo)	30 / 110	0.273	$[0.198, 0.363]$	0.500	$20\text{W}/10\text{L}, p = 0.10$
`rank2`	**Forbid opening tokens of the bad value via logit mask**	**35 / 110**	0.318	$[0.239, 0.410]$	0.000	$22\text{W}/7\text{L}, p = 0.008$
`oracle_force`	Inject ground-truth value at t^*	94 / 110	0.855	$[0.777, 0.908]$	—	Reference (79.5% non-trivial)

3.1 The Greedy No-Op and Prompting Placebo

1. **Greedy Replay Fails:** `rewind_greedy` reproduces the identical erroneous digit in 61.8% of instances, achieving a lower recovery rate than a fresh rewrite (0.155 vs 0.182). Slicing back to the error point without perturbation is mathematically inert.
2. **The Hindsight Prompt is an Inert Perturbation:** Natural language hindsight constraints (`# constraint: e is not 7`) recover exactly $30/110$ chains (0.273). However, the neutral placebo line (`# constraint: none`) **also recovers exactly $30/110$ chains (0.273)**. In a direct paired comparison, the real constraint and the placebo tie:

$$30/110 \quad \text{vs} \quad 30/110 \quad (15\text{W}/15\text{L},\ p = 1.000)$$

The prompt text suppresses the named token (re-emission drops to 13.6%), but does not steer generation toward truth; the model selects an alternative erroneous digit instead. A plausible reading is that the apparent benefit comes from the extra prompt tokens rather than from the constraint, but the experiment does not separate whitespace, token shift and constraint content, so that attribution is not established.

3.2 Rank-2 Logit Exclusion Beats Sampling

Physical logit masking (`rank2`) sets the logits of the tokens that could begin the rejected value to $-\infty$:

$$\tilde{z}_i = \begin{cases} -\infty, & \text{if } i \in \text{FirstTokens}(\text{bad_val}) \\ z_i, & \text{otherwise} \end{cases}$$

This prevents the model from beginning the next value with the same token; it does not enumerate alternate complete answers or select the globally second-best chain. `rank2` recovers 31.8% of broken chains ($22\text{W}/7\text{L}, p = 0.008$), eliminating re-emission of the *same* value (0.0%). Because the exclusion applies to the opening digit, a different multi-digit value in the same digit range remains reachable, which is consistent with the gap to the oracle-injection arm in Table 2.

---

4. Repair Conditional on Known Failure and Measurement Audit

We wired the Two-Tier localizer, the `rank2` intervention, and an "Accept-if-Cleaner" gate into an automated rewind loop. The loop is not given the error index at inference time, but every chain it is run on was already selected as complete and broken using ground-truth `first_error`. The measurements below are therefore repair conditional on known failure, not detection (§3).

4.1 The Accept-if-Cleaner Gate

When an observer mis-localizes a late defect to Step 1, applying `rank2` risks corrupting an already-correct prefix. We implemented a confidence guard:

- Let lp_{orig} and lp_{edit} be the mean token log probabilities of the original and repaired value strings, respectively, evaluated under the unmasked base model with the same prefix. Masking applies during candidate generation, not during these readouts.
- If the original step was confident ($lp_{orig} > -0.25$) and the edit sharply degrades confidence ($lp_{edit} < lp_{orig} - 0.35$), the edit is **rejected**, and the prefix is preserved.

In empirical holdout testing, the gate rejected $23/110$ candidate edits. The gated and ungated Two-Tier arms recover the same number of chains ($32/110$), because the rejected rows contributed zero recoveries when ungated. On the vault, $81/331$ edits were rejected with 0 recoveries among them, and only the gated arm was run there, so no ungated vault comparison exists. On this sample the gate is a *conservative filter that did not reduce recovery*, not a demonstrated recovery gain: it removes edits that would not have helped and buys precision ($32/87 = 36.8\%$ recovery among accepted rows on $N = 110$). Their identical token and time totals are shared accounting, not a measurement: the repair timer is reused across both arms and the gate is evaluated after the candidate has already been generated, so those totals are not evidence that gating was free. An auxiliary probe evaluated falling back to the localizer's second-highest ranked step ($\hat{t}_2$) on those 23 rejected rows: fallback hit the true step in $3/23$ instances but converted $0/23$ chains into additional recoveries, which is why terminating on rejection is the compute policy we use.

Table 3: Conditional Repair Recovery on the Tuning Holdout ($N = 110$)

Autonomous Arm	Localization Model	Intervention	Recovered / N	Success Rate	95% Wilson CI	$\hat{t} = t^*$ Exact	Accepted / N
`full_rewrite`	None (Sample $T = 0.8$)	Full sequence	14 / 110	0.127	$[0.077, 0.202]$	—	—
`linear_ungated`	Per-step MLP Majority	`rank2`	24 / 110	0.218	$[0.151, 0.304]$	0.618	—
`ensemble_ungated`	Ensemble Average	`rank2`	31 / 110	0.282	$[0.206, 0.372]$	0.691	—
`two_tier_ungated`	**Two-Tier Rule**	`rank2`	**32 / 110**	0.291	$[0.214, 0.382]$	0.718	—
`ensemble_gated`	Ensemble Average	`rank2` + Gate	31 / 110	0.282	$[0.206, 0.372]$	0.691	66 / 110
`two_tier_gated`	**Two-Tier Rule**	`rank2` + **Gate**	**32 / 110**	0.291	$[0.214, 0.382]$	0.718	**87 / 110**

Autonomous loop recovery rises from 12.7% (rewrite) and 21.8% (per-step MLP) to 29.1% ($32/110$). Under an omniscient oracle that intervenes at the true step, `rank2` recovers $35/110$ (31.8%, Table 2). The autonomous loop therefore reproduces $32/35$ of that observed oracle-localized count on this tuning sample:

$$\frac{32}{35} = \textbf{91.4\%} \text{ of the observed oracle-localized rank-2 recovery count}$$

This is a descriptive ratio between two arms measured on the same 110 rows, not an upper bound. 31.8% is a count for one intervention with one decoding rule, not a ceiling over interventions: a different edit, an earlier rewind, or a stronger decoder could in principle exceed $35/110$. The oracle was not re-measured on the vault, so no vault ratio is claimed. The vault result (Section 4.3) stands on its own paired comparison against full rewrites.

4.2 Historical Cache-Slicing Measurements (Not Equal-Work Speedups)

An earlier version of this section reported a $25.9\times$ generation-slice speedup and a $2.0\times$ end-to-end loop speedup from physical key-value cache truncation via `DynamicCache.crop`. Both interpretations are withdrawn.

The timed block builds a fresh cache over a *prefix* prompt, calls `crop` with that cache's current length, and stops the timer before any repaired suffix token is generated (`experiments/eval_vault.py:377-385`, `experiments/path_a_loop.py:269-278`). Cropping a cache to the length it already has is a no-op, and the rewind that matters — truncating a cache that holds a longer, failed trajectory — is never performed inside the measurement. The recovery outputs are produced separately by `gen_step` and `finish_chain`, which pass fresh prompts to `model.generate` and never receive the cropped cache (`experiments/oracle_rewind.py:93-117`). The token counters add $(steps - \hat{t}) \cdot 2$ without generating those tokens inside the timed block.

The saved record therefore contains a prefix-prefill fragment, not cached repair generation:

Table 4: Historical Prefix-Prefill Record ($N = 110$ Chains, GTX 1050 4GB)

Timed operation	Formula token estimate / chain	Recorded time / chain
Full fresh-prompt rewrite	581.1	4,498.4 ms
Fresh-prompt repair (ensemble target)	379.5	2,732.2 ms
Prefix prefill plus equal-length crop; no suffix generation	119.7	173.7 ms

The rows time different work. No relative speedup is inferred.

These values are retained because they are stored in `results/path_a_loop_h5.json` and the systems block of `results/verify_vault_h5.json`, and because a correction should not erase history. They do not support a speedup claim:

- **No longer cache is truncated.** The cache being cropped was built from a short prefix and cropped to its own length.
- **No repaired suffix is timed.** The timer stops before generation; the $73/331$ vault recoveries were produced outside it by fresh-prompt generation.
- **No observer or gate cost is included.** Feature extraction, localizer forward passes and the gate readout fall outside the timed region.
- **Tokens are formulas.** Each arm's token count is computed as prompt length plus two per step, not measured from model forwards, so the -79.4% is an estimate of a quantity that was also not generated in the timed block.

The "$2.0\times$ loop" and "$1.70\times$ vault loop" figures are withdrawn for the same reasons. The per-chain value $246{,}826\ \text{ms}/110 = 2{,}244\ \text{ms}$ does come from a timer, but that timer excludes the observer and gate, and the gated and ungated arms share one repair timer. Their identical cost is shared accounting, not a measured equality.

What would be needed. A defensible benchmark would build a cache over a full failed trajectory, truncate it to a token boundary strictly inside that trajectory, generate and verify the repaired suffix on the truncated cache, and price feature extraction, localization, repair, gating and verification inside one timer, with the baseline required to produce the same output work. The independent cache evidence this project does have is numerical rather than temporal: on a CPU float32 run, cropped-cache continuation matched fresh recomputation to $\max|\Delta\text{logit}| = 1.86 \times 10^{-5}$ with matching argmax (`results/arc_kaggle_scaffold_smoke/verification_results.json`). That supports coherence of the operation, not a speedup.

4.3 Vault Evaluation ($N = 331$ Broken Candidates)

To examine transfer beyond the tuning split on small holdout splits, we conducted a blind vault replication on an independent test split (`seed=20261028`, 140 tasks, 560 candidates, yielding 331 **complete broken chains**) with the training recipe and thresholds fixed before vault evaluation; the scripts re-fit the heads on the original training set and hold them fixed during evaluation ($\tau_{trf} = 0.35, lp_{orig} = -0.25, \Delta lp = -0.35$).

Table 5: Blind Vault Replication and Permutation Control ($N = 331$ Unseen Chains)

Generation Arm	Intervention Mechanism	Recovered / N	Rate	95% Wilson CI	$\hat{t} = t^*$ Exact	Accepted / N
`full_rewrite` ($T = 0.8$)	Regenerate entire chain	46 / 331	0.139	$[0.106, 0.180]$	—	—
`random_t_rank2`	Uniform random $\hat{t} \in \{1 \dots 4\}$ + `rank2`	23 / 331	0.069	$[0.047, 0.102]$	0.245	—
`two_tier_gated` **(CGR)**	**Two-Tier Probe + Gate +** `rank2`	**73 / 331**	0.221	$[0.179, 0.268]$	0.625	**250 / 331**

1. **Localization Transfers; the Hard Threshold Over-Triggers (Rule 1, reframed):** The descriptive complementarity holds out of sample: Transformer Majority Top-1 reached 0.677 (vs 0.604 for Linear), up from 0.618 on the tuning holdout, and the Ensemble Average held at 0.680 — a 0.011 drop, not a collapse. The **Two-Tier rule** lands at 0.625 Top-1, below both, and the reason is not that it stopped finding tail errors. It is that $\tau_{trf} = 0.35$ is a recall lever: it calls Step 4 on 68 **of** 331 **rows where only** 33 **are Step 4**, buying the highest Step-4 recall of any arm at the lowest Step-4 precision.

Arm	Top-1 (published fit)	Step-4 Recall (re-fit)	Step-4 Precision (re-fit)	Rows called Step 4 (re-fit)
Per-step MLP Majority	0.604	0.394	0.342	38
Transformer Majority	0.677	0.545	0.400	45
Ensemble (softmax mean)	0.680	0.485	0.471	34
Two-Tier ($\tau = 0.35$)	0.625	0.667	0.324	68

$\tau_{trf} = 0.35$ was selected on the same 110 rows where it scored 0.812 Step-4 recall and 0.718 Top-1; on unseen seeds it is calibration drift, not a law. The transferable finding is the *complementarity* of linear and cross-step attention, and the fact that a hard cutoff converts it into a precision/recall trade rather than a free gain. The ensemble keeps the complementarity without the cutoff.

Metric erratum (2026-10-29): the previous version of this item read "0.625 / 0.410 Step-4 recall, below both". The 0.410 was a per-step **precision**, produced by calling `per_step_recall(lab, pred)` where the signature is `per_step_recall(pred, lab)`. On recall the Two-Tier arm was never below the others; it was above them. Top-1 above is from the published fit because that fit drove the loop in Table 5; the recall, precision and call-count columns are a deterministic CPU re-fit (two runs identical), because the 2026-10-28 fit ran on a different torch build and stored no predictions. The re-fit

reproduces Linear and Transformer Top-1 within 0.01 and moves the two threshold-dependent arms by 0.027 (Two-Tier $0.625 \rightarrow 0.598$), which is the same `P_trf(4) >= 0.35` fragility argued for here. Both fits agree on the ordering that matters: Two-Tier maximizes Step-4 recall and minimizes Step-4 precision. See Errata.

2. **Defeating the Permutation Control (Rule 3 PASS):**
   Slicing back to a uniformly random step with rank-2 exclusion (`random_t_rank2`) collapses to 6.9% **recovery** $(23/331)$—performing at roughly half the recovery rate of doing a standard full rewrite (13.9%). This particular random-step policy recovered fewer already-broken chains; damage to initially correct chains was not measured. In contrast, Two-Tier CGR achieves 22.1% $(73/331)$. The candidate-level paired test gives the following result; within-task dependence is not accounted for:

   $$\textbf{57}\ \text{Wins}\ /\ \textbf{7}\ \text{Losses} \quad (p = 7.64 \times 10^{-11})$$

   The paired odds ratio of $8.14:1$ shows a recovery advantage over this particular random-step control; it does not isolate all causal mechanisms.

3. **Vault timing interpretation withdrawn:** The historical 3,281.9 ms rewrite time and 120.7 ms prefix-prefill fragment do not measure equivalent work. The 1,934.7 ms repair timer excludes localization and gate evaluation. None establishes a cache-generation or full-loop speedup. Formula-derived token totals are not measured FLOPs or model-forward counts (§4.2).

4. **Cache-coherence evidence:** A historical findings report describes 20 comparisons with maximum logit difference 4.86e-5 and matching argmax outputs. The current audit has not located an equivalent per-case machine-readable record for that older check, so it is not treated as independently verified here. The separately saved CPU smoke test checks one fixed prompt, with maximum difference 1.86e-5 and matching next-token argmax; it does not establish all-token equality, GPU behavior or speedup (§4.2).

4.4 Longer-Horizon Development Evaluation ($H = 10$)

To establish the limits of test-time KV rewinds as reasoning chains deepen, we scaled the arithmetic task from $H = 5$ to $H = 10$ assignments on `Qwen3-0.6B-Base` ($N = 75$ broken holdout chains, `results/verify_h10_scaling.json`).

Table 6: Horizon Scaling Diagnostics and Recovery at $H = 10$ ($N = 75$)

Evaluation Arm	Mechanism	Top-1 Localization	Full-Chain Recovery	95% Wilson CI	Formula token estimate
`linear_localizer`	SVD-32 per-step MLP (5 seeds)	0.520 (Chance 0.100)	—	—	—
`full_rewrite`	Regenerate from Step 0 ($T = 0.8$)	—	0 / 75 (0.000)	$[0.000, 0.049]$	$1,656.2$
`in_context_oops`	Verbal retry prompt without KV crop	—	0 / 75 (0.000)	$[0.000, 0.049]$	220.6
`two_tier_gated_1rewind`	Two-Tier CGR + `rank2` + Gate	0.520	0 / 75 (0.000)	$[0.000, 0.049]$	165.0
`two_tier_gated_max2`	Sequential max-2 rewinds	—	0 / 75 (0.000)	$[0.000, 0.049]$	171.9

The tested policies recovered 0/75 complete broken chains at H=10. This does not locate a horizon boundary at 6–8 steps, establish an impossibility result, or identify model capacity as the cause. Localization, intervention quality, task difficulty and downstream errors remain possible contributors. Tools and other

model checkpoints are future comparisons, not demonstrated necessities.

The historical 165.0 versus 1,656.2 token figures are formulas, not instrumented forward counts. The H=10 repair implementation uses fresh prompts and does not crop a persistent cache, so the previously claimed 90% cache-related token saving is withdrawn. The saved 6,399.2 versus 7,502.7 ms timings cover different generation policies; the repair includes gate readouts but excludes localization and feature acquisition. They do not establish a cache or full-controller speedup.

---

5. Limitations & Remaining Headroom

The recovery observations are conditional on known failures, not a complete detection-and-repair evaluation. Historical cache speedups and full-loop timings are withdrawn (§4.2). Candidate-level statistical tests do not account for multiple completions of the same task. Other limitations follow.

1. **The Observed Rank-2 to Oracle Gap (31.8% vs 79.5%):** Under an oracle that directly injects the ground-truth value, 79.5% of non-trivial broken chains are restored. The tested `rank2` arm recovered 31.8% on this sample. First-digit exclusion guarantees that the model will not repeat its previous answer, but when numbers span multi-digit ranges ($[1, 99]$), excluding one digit still leaves multiple plausible incorrect candidates. Closing this remaining headroom requires token-level value steering or beam-guided branch exploration at step $\hat{t}$.
2. **Longer-horizon failure:** None of the tested H=10 policies recovered a complete chain. The data do not establish a universal horizon or that only tools or larger models can improve it (§4.4).
3. **Hybrid Architecture Compatibility:**
   In linear-attention or state-space models (e.g., Mamba, LFM3), key-value caching is replaced by recurrent convolutional and state matrices. Rewinding in such architectures requires rolling back the hidden recurrent state h_t rather than slicing a sequence cache.

5.1 Scaling Hypothesis: Longer Tasks and Different Models

The observed $H = 10$ recovery failure motivates studying whether stronger generators, external tools or different repair policies can extend the usable horizon. Model capacity is one plausible contributor, but the current experiments do not identify it as the sole cause; localization errors, intervention quality, task difficulty and accumulated downstream errors may also matter. Neither successful large-model repair nor a universal horizon threshold follows from the present measurements.

A future evaluation could compare full rewrite, checkpointed continuation, localized base repair and specialist-assisted repair on the same long-context tasks across model sizes, while measuring verifier quality, retained-prefix correctness, full-loop latency, memory use and task success. Context-retrieval or experience-memory interventions could be tested separately with controls, so an improvement is not attributed to cache truncation alone. The existing arithmetic results and isolated adapter bridges are useful tests of particular mechanisms, not evidence that the complete approach will transfer unchanged to frontier systems.

Cache operations and adapter switching depend on model architecture, tokenizer, cache representation and runtime. A LoRA adapter trained for one backbone cannot simply be attached to a different-size backbone, and caches from different models or policies must not be assumed compatible. Full specialist prefill or a separately validated bridge is required where applicable. Any claimed memory capacity, loading latency, end-to-end speedup or benefit at larger scale must be measured on the target setup.

---

6. Dynamic Adapter Routing and the Bridge Protocol

> **Status: preliminary coder integration and single-question cyber routing; multi-domain DAG efficacy pending.** The CPU prototype measures coder switching and base Bridge coherence on disclosed tasks (§6.3), and a learned cyber correctness router in a separate single-question experiment (§6.5). It does not establish multi-domain DAG recovery, deployment speedup, or a 4 GB memory bound.

Full-checkpoint escalation was measured and closed. On the $N = 110$ ladder no deployable actuator beat `rank2` (`findings/2026-10-28-level2-actuator-ladder.md`), and the memory arithmetic is unforgiving on the target hardware: the AR backbone occupies ∼1.2 GB and a 1.5–2.5B-class specialist a further ∼2.2 GB, so a two-checkpoint policy spends ∼3.4 GB before activations on a 4 GB budget. The alternative is to swap low-rank adapter weights (∼10–50 MB) instead of a parameter backbone. The CGR line already produces adapters in that band — the matched LoRA control trains 22.3M parameters (`findings/2026-10-25-steering-capacity-control.md`), ≈ 45 MB at fp16. A separate coder LoRA checkpoint is now available on the chat-capable `Qwen/Qwen3-0.6B`, rather than the arithmetic experiments' `Qwen3-0.6B-Base`. Its preliminary integration is reported in §6.3; specialist quality is a separate question from its size or successful loading.

6.1 KV-Cache Coherence Under Adapter Intervention

A fundamental systems constraint in recurrent adapter intervention is **Key-Value Cache Coherence**: updating attention projections (ΔW_q, ΔW_k, ΔW_v) or feed-forward layers (ΔW_{MLP}) alters the activation distribution, rendering cached tensors mathematically misaligned with subsequent decoding steps. Two operational paths can manage this boundary.

Path 1 — Query-Restricted Adaptation (in-stream persistence). Adapters are constrained during training to low-rank perturbations of query and MLP projections (ΔW_q, ΔW_{MLP}), keeping the key-value projection weights (W_k, W_v) strictly frozen. When an error is localized at index $\hat{t}_{\min}$, the controller crops the cache via `DynamicCache.crop` to $\hat{t}_{\min} - 1$. Because W_k and W_v are invariant, this design leaves their projection matrices unchanged. It does **not** imply cache equivalence: Q/MLP changes can alter the hidden states from which subsequent layers compute keys and values. Attending to a base-produced prefix under specialist weights is therefore a mixed-policy computation that must be compared with full-prefix specialist prefill. Keeping the adapter active to termination avoids another switch but does not itself prove coherence or equivalent generation quality.

The separately saved CPU smoke check compares cropped continuation with fresh prefill on one prompt (maximum logit difference 1.86e-5; next-token argmax matches). It uses base weights only, so it does not establish equivalence of mixed base/adapter cache policies. Historical 20-sequence numerical claims need their original per-case artifact before being treated as independently audited.

6.2 The Bridge Protocol: Targeted Re-Encoding

When a specialised adapter is needed only for a localized sub-routine — repairing code or symbolic arithmetic at steps $\hat{t} \dots \hat{t} + k$ — before returning to general causal generation, retaining adapter overhead across the remaining sequence is wasteful. The **Bridge Protocol** treats adapter generation as an isolated, temporary patch:

1. **Surgical truncation.** The cache is cropped to the recorded absolute token boundary before step $\hat{t}$, preserving verified prefix representations. A clean base-prefix copy is retained separately from the specialist's cache.

2. **Specialist execution.** The task-specific adapter is engaged to generate the localized patch tokens $S_{\text{patch}} = \{x_{\hat{t}}, \ldots, x_{\hat{t}+k}\}$.
3. **Distribution re-alignment (the bridge).** The adapter is disengaged. Rather than flushing the context window or retaining adapted latent vectors, the raw token IDs of S_{patch} pass through a non-generative forward prefill under the frozen base weights, starting from the preserved base-prefix copy. The specialist's working cache is discarded. This rebuilds patch keys and values under the base policy.
4. **Resumption.** Autoregressive decoding resumes under the base policy at step $\hat{t} + k + 1$ from the bridge's output logits, without feeding the final patch token twice. Dependent artifacts downstream of a repaired node must be revalidated or regenerated. Independent artifacts may be preserved, but their later cached states must be replayed if their prior attention context included the changed prefix.

The economic argument is that prefill over a short window is parallel, so re-encoding k tokens costs one forward pass rather than k sequential decode steps, and the specialist's weights can be de-allocated immediately. That argument is plausible but **its cost is not established here**, and Section 4.4 cuts against the optimistic reading: at $H = 10$ a 90% reduction in forwarded tokens bought only $1.17\times$ wall-clock, because launch and prefill overhead dominate at this scale. Any claim that the bridge is "imperceptible" or "single-digit milliseconds" would therefore contradict a measurement already in this paper. It has to be timed.

What would count as evidence. The protocol is worth adopting only if, against query-restricted persistence (Path 1) on the same rows, it satisfies all three:

1. **Cost:** re-encoding + adapter swap is cheaper in wall-clock than keeping the adapter resident for the remaining $T - (\hat{t} + k)$ steps. Tokens forwarded is the wrong metric; Section 4.4 shows it decouples from latency.
2. **Quality:** end-to-end recovery is not lower. The bridge rewrites the patch into the base distribution, so the failure mode to watch is a specialist edit that the base model then treats as out-of-distribution and talks itself out of.
3. **Coherence:** $\max |\Delta\text{logit}|$ between bridged and fully-base decoding over the same token sequence satisfies a predeclared, dtype-specific tolerance, with finite logits and matching continuation. The original 4.86×10^{-5} is an observed base-only maximum, not a universal adapter tolerance. The CPU fp32 integration below uses 10^{-4}.

If (1) fails, the protocol collapses to Path 1 with extra machinery. If (2) fails, the adapter was load-bearing for more than the patch and targeted re-encoding is the wrong abstraction. Either outcome is a result; neither is currently known.

6.3 Preliminary Coder Integration and Validation Readiness

A CPU fp32 prototype connects the existing commitment controller to one coder LoRA (`checkpoint-1000`, parent revision `c1899de289a04d12100db370d81485cdf75e47ca`). Task skills and deterministic verifiers are supplied explicitly; no learned code localizer or router is used. In a five-node base/coder/base/coder/base sequence, four policy transitions were verified across 112 LoRA layers and four nodes were committed. The coder failed the clamp task (2/5 function cases), which the controller rejected; the even-sum task passed (4/4 cases) and was bridged back to the base. The base diagnostic passed both coding tasks. These observations establish neither a specialist quality gain nor the cause of the clamp regression.

All accepted-node Bridges in that sequence had exact retained-prefix K/V, finite logits, matching next-token argmax, correct cache length, and matching four-token greedy continuations against fresh base prefill. The largest absolute logit delta was 5.20×10^{-5} against the predeclared fp32 tolerance 10^{-4}. An explicitly injected bad addition function was also rejected, repaired by the actual coder, and bridged successfully. That

control tests the mechanism; it is not natural base-failure recovery. Source records: `results/cgr_adapter_switching/20261029T213116Z_df30837b.json` and `results/cgr_adapter_switching/20261029T213212Z_4e3f62d0.json`.

A separate dependency/validator package exercises delayed failure, descendant invalidation, preserved-artifact replay planning, parameterized SQL behavior, and static HTML contracts on 20 public parameter variations of one sanitized-card task family. Its passing fixture checks use zero model generations and supply no multi-domain efficacy evidence. Real frontend behavior, multi-adapter cache tests, independent task evaluation and the full-prefix-versus-switched-specialist ablation remain pending. The prospective protocol is `ADAPTER_VALIDATION_PROTOCOL.md`.

6.4 Two-Specialist Dependency-Graph Development Test

A live four-node derived card fixture uses one pinned chat-capable `Qwen/Qwen3-0.6B` backbone with both cyber and coder adapters. Its generated cyber SQL passed the parameterized-query contract; the result's row count was then checked using a generated coder factorial function. Both artifacts were committed to `DependencyGraph` after verification and base-bridge comparison, and the final composed check passed. The cyber and coder bridges differed from fresh-base reference logits by at most 4.863739×10^{-5} and 3.337860×10^{-5} respectively, below the fp32 tolerance of 10^{-4}; argmax and four-token continuations matched. The 31.947-second CPU time includes base diagnostics, specialist attempts, verifiers and reference checks; initial model loading took 7.855 seconds separately. This is one disclosed development example, not a latency comparison.

The base also passed both node contracts in that sequence, so it does not demonstrate that specialists were necessary.

A second four-node graph used the original `card_00` clamp and defensive SQL contracts. The base passed clamp; the coder initially failed but passed after one retry whose prompt contained the verifier's failed cases. The cyber query then passed in the accumulated conversation, both exact-output base bridges passed with maximum logit differences 5.245209×10^{-5} and 5.483627×10^{-5}, and the final check consumed both outputs: a SQLite row for Alice and the correctly clamped value -10. The wall time was 25.229 seconds excluding 10.209 seconds loading, including diagnostics, the failed attempt and verifier feedback. This is a disclosed coder-failure retry, not a natural base-failure repair, a blind evaluation or the original 20-node delayed-failure/replay milestone.

An independent factorial task produced a genuine base error and a coder output passing three predefined cases; its accepted-token base bridge differed from fresh prefill by at most 4.434586×10^{-5}. However, attempting the cyber task after this repair failed: the cyber adapter repeated the code instead of producing JSON. On the initial no-feedback `card_00` clamp attempt, the base passed and coder failed, so that attempt was rejected. A bounded eight-task search after the cyber bridge found three base failures and no verified coder repairs. The successful derived graph is not the original 20-node delayed-failure fixture or evidence of a complete multi-domain recovery; the arithmetic experiments elsewhere still use the separate `Qwen3-0.6B-Base` backbone. Full records and timing definitions are in `findings/2026-10-30-coder-cyber-bridge.md`.

6.5 Cross-Dataset Cyber Routing and Live Full-Prefill Verification

We trained binary correctness heads on a 200-item single-answer SecEval sample, where the base answered 124 correctly and 76 incorrectly. The inputs were the final four generated answer-token hidden states, with standardization and an eight-dimensional SVD fitted solely on the training set. A `LinearStep` and a `TransformerStep` were adapted to binary classification; the mean of their sigmoid scores below 0.50 selected the cyber adapter. The heads were trained, but the base and the step-2500 cyber LoRA were frozen during inference. The score is not established as a calibrated correctness probability.

CyberMetric-80 arm	Correct	Accuracy
Base (Qwen3-0.6B)	54/80	67.50%
Cyber adapter	54/80	67.50%
SecEval-trained router	**57/80**	**71.25%**
Oracle, choose a correct arm if available	61/80	76.25%

The router selected the adapter on nine questions, producing three useful switches and zero harmful switches (two-sided exact paired binomial p=0.25 versus base). These are sample-level observations, not a proof of safety. On the disjoint 40-question SecEval sample, base/cyber/router/oracle scored 26/23/26/29; one switch helped and one harmed. In the reverse direction, heads trained on all 80 CyberMetric items evaluated on 100 SecQA items gave base/cyber/router/oracle scores of 97/90/97/97, with nine switches and neither useful nor harmful switches. Strict answer-format parsing counted 2 CyberMetric, 5 SecEval and 7 SecQA cyber responses as invalid; those must not silently be treated as correct.

The live CyberMetric loop regenerated the base output, computed the frozen router score, and, when selected, discarded the previous generation cache and generated from the complete prompt with the cyber adapter. It reproduced all 80 saved base outputs and routing decisions and all nine selected standalone cyber outputs. A forward hook verified that each specialist's initial forward received the full prompt and a cache of length zero. This demonstrates the specified full-prefill behavior and outcome on these questions. It does not verify a multi-step DAG, classifier calibration, safe commit decision, or specialist-to-base bridge. The SecEval and CyberMetric material is public and CyberMetric had already been examined; exact and high character-similarity checks do not eliminate paraphrases or unknown pretraining/adapter contamination. The +3/80 gain is exploratory and does not establish generality or safety. Artifacts: `experiments/cyber_router_crossdataset.py`, `experiments/cyber_router_live.py`, `results/cyber_router_crossdataset_v1.json`, and `results/cyber_router_live_cybermetric80_v3.json`.

6.6 Experience Memory and UI: Exploratory Results

An in-memory Chroma store with MiniLM embeddings retrieved a short formatting rule for a 20-item CyberMetric sample. The cyber arm moved from 13/20 with two format failures to 16/20 with none; the base moved from 15/20 to 16/20. The rule was written after observing those failures, and the other questions had been examined, so this does not establish held-out generalization or an advantage of retrieval over directly including the same instruction. Scoped, expiring memory was tested as a component, not integrated into the live routing loop.

Experience Memory as a Semantic Bridge: Controlled Development Test

We investigated whether a scoped experience rule could help the cyber adapter follow its output contract after seeing coder-generated text. We reconstructed three previously verified coding contexts (factorial, clamp with verifier feedback, and even-sum) and combined each with two related SQLite tasks (selecting a user by name or ID). The cyber adapter used fresh full-prompt prefill in every condition, with the same greedy settings. The memory rule, written after observing the factorial failure, instructed the model to output only a JSON object containing `sql` and `params`. Chroma retrieved the rule using MiniLM embeddings within an explicitly supplied task scope; the memory tag was inserted as text in the user message, not as a privileged system instruction.

Condition	SQL contract passed	Bare JSON and contract passed	Repeated Python
Cyber task alone	6/6	0/6	0/6
After coder output, no rule	2/6	2/6	4/6

Condition	SQL contract passed	Bare JSON and contract passed	Repeated Python
After coder output + direct rule	6/6	6/6	0/6
After coder output + retrieved rule	6/6	6/6	0/6

The standalone outputs passed the SQL behavior contract after removal of an outer Markdown fence but did not satisfy strict bare-JSON formatting. The same two standalone prompts were repeated across the three context rows, so 6/6 is not six independent standalone tasks. Without the rule, the factorial and even-sum contexts led to repeated Python on both SQL tasks; the clamp-feedback context did not. Adding the rule was associated with passing outputs on these development cases, but the study did not isolate whether code content, chat structure, context length or another factor caused the initial failures.

The direct and retrieved conditions had identical prompt token IDs and identical outputs in all six comparisons. **This demonstrates scoped retrieval and application of a manually authored rule in these cases; it does not establish that the memory system automatically detected the error, discovered the rule, or outperformed directly supplying it.** The rule was created after the observed failure, scope was supplied by the caller, and the six combinations are not an independent held-out benchmark. A future semantic-bridge study should evaluate multiple candidate rules, irrelevant and expired memories, and unseen task families. Complete inputs, outputs and checks are in `results/cyber_context_memory_controls_v1.json`; the protocol and limitations are in `findings/2026-10-30-cyber-context-memory-controls.md`.

Five-rule ablation. We next stored five handcrafted SQL, Python, math, UI and phishing rules in the same supplied scope and queried Chroma with the current task text. The SQL rule ranked first for all three SQLite prompts; four hand-authored non-SQL control queries also selected their intended rule. After the saved coder factorial output, the no-memory cyber arm passed 0/3 SQL checks and the retrieved-rule arm passed 2/3. The remaining output was valid JSON but selected a non-existent `user_id` column; its reworded prompt described "user id" without explicitly specifying the `id` schema, so the failure is ambiguous. These are three related development prompts, not independent held-out tasks.

A poem query with no active scope retrieved no rule. This isolation depended on the caller-supplied scope: when the same poem was queried within the five-rule scope, top-1 retrieval returned the irrelevant math rule under the then-ungated implementation. Thus the negative control does not establish automatic abstention or context-contamination detection. The prior direct-instruction comparison remains essential: matching retrieved and direct prompt tokens produced matching outputs. See `findings/2026-10-30-multirule-memory-ablation.md` and `results/multirule_memory_ablation_v1.json`.

Strict similarity gate. A subsequent implementation adds the requested rule that a retrieved candidate must have cosine similarity strictly greater than 0.90. With this cutoff the poem query was rejected even when the five-rule scope was supplied, and an exact match to the stored SQL description was retained. However, all three ordinary SQL prompts were also rejected: their relevant-rule similarities were approximately 0.575, 0.568 and 0.642. The four other domain-control queries also fell below 0.90. This illustrates the false-abstention cost of an uncalibrated cutoff; cosine similarity is not a probability of correctness and can range from -1 to 1. The gate-only check did not generate new model outputs, so the earlier ungated generation table remains a separate experiment. The threshold defaults to 0.90 and can be configured; the earlier scripts explicitly use an ungated setting to reproduce their original protocol. See `results/multirule_memory_gate_090_v1.json`.

A paired FocusFlow landing-page prompt produced complete HTML from both the base and the final UI adapter, with identical prompt tokens and decoding settings. Both omitted the requested functional signup, working hero action, schedule preview and color palette; the adapter output also lacked an associated email

label and its mobile layout expanded beyond the target width. One example is not a UI benchmark. The UI data's React/Next.js component emphasis is a plausible mismatch with the self-contained HTML task, not a demonstrated cause.

---

7. Conclusion

On complete arithmetic chains already known to be wrong, the recorded localizer-guided token-exclusion policy recovered 73/331 chains versus 46/331 for full rewrite. This is a conditional repair result from fresh-prompt generation; the historical implementation does not connect these outcomes to persistent cropped-cache decoding. Task clustering limits the candidate-level statistical interpretation.

Separate numerical checks support cache-continuation coherence in tested configurations. The historical timing blocks do not establish a cache or end-to-end speedup. The live cyber router reproduced 57/80 correct answers versus 54/80 for each individual arm, with three useful and zero harmful switches in this sample (exact paired p=0.25). Multi-domain bridge examples and memory controls remain development demonstrations. A unified, independently evaluated controller and an equal-work systems benchmark remain future work.

What this paper does not establish. Four boundaries are load-bearing, and each is stated where the relevant experiment appears:

1. **No live 20-node delayed-failure graph.** The dependency-graph results in §6.3–§6.4 are four-node development fixtures with deterministic verifiers; the original 20-node detect/invalidate/replay milestone has not been run.
2. **No claim that specialists outperform the base.** In the two-specialist sequence the base passed both node contracts; the coder failed the clamp task (2/5 cases) while the base passed (5/5); and an eight-prompt search after a cyber bridge found three base failures and zero verified coder repairs (§6.4).
3. **No unified multi-domain DAG.** Every arithmetic result uses `Qwen3-0.6B-Base`; every adapter, bridge and routing result uses the chat-capable `Qwen/Qwen3-0.6B` at revision `c1899de289a04d12100db370d81485cdf75e47ca`. No experiment spans both backbones, so the system is an integration of separate experiments, not one measured pipeline.
4. **No measured multi-checkpoint memory bound.** The recorded peaks — 2,293.5 MB and 2,328.0 MB — are single-0.6B-checkpoint runs. The ~3.4 GB two-checkpoint figure in §6 is arithmetic from checkpoint sizes before activations, not a measured peak.

The routing gain is +3/80 on a benchmark that had been examined before the experiment, contamination is unknown, and fp16 Bridge tolerance is untested.

References and research artifacts

- Tihanyi et al., CyberMetric: A Benchmark Dataset Based on Retrieval-Augmented Generation for Evaluating LLMs in Cybersecurity Knowledge, IEEE CSR 2024; dataset.
- XuanwuAI, SecEval cybersecurity benchmark and dataset.
- Liu et al., SecQA: A Concise Question-Answering Dataset for Evaluating Large Language Models in Computer Security; dataset.
- Qwen team, Qwen3-0.6B chat model and Qwen3-0.6B-Base.
- Project source records and model/data revisions are indexed in `REPRODUCIBILITY.md` and in the linked findings and result files.

Errata

2026-10-30 — systems timing and method audit. The 25.9x/27.2x cache-generation and 2.0x/1.70x full-loop interpretations are withdrawn. The cache block timed prefix prefill without suffix decoding; repair timers excluded observer and gate costs. Token/FLOP savings were formulas. `LinearStep` is an MLP, not a linear probe; its arithmetic input differs from the Transformer's. No universal 6–8-step horizon or theoretical rank-2 ceiling is established. Raw historical JSON is preserved; `findings/2026-10-30-paper-review.md` records the evidence and `results/measurement_audit_20261030.json` records machine-readable caveats.

2026-10-29 — the vault Step-4 column was a precision, not a recall. `experiments/eval_vault.py` called `per_step_recall(v_lab, pred)` and `balanced_accuracy(v_lab, pred)` where both signatures are `(pred, lab)`. Every `*_rec` array written to `results/verify_vault_h5.json` is therefore per-step *precision*, and every `*_bal` value is a macro-precision. The consequence is the sentence this paper used to carry — that the Two-Tier rule fell to `0.410` Step-4 recall and underperformed both single probes — which is the opposite of what the arm does on recall.

The call order is fixed; `per_step_precision` is now a separate function so the two metrics cannot be conflated again; and `eval_vault.py --diagnostics-only` recomputes the localizer block without touching the stochastic arm results, refusing to splice a drifted fit over the original. The corrected block is `results/verify_vault_h5_localizer_refit.json`; `results/verify_vault_h5.json` is annotated in place and **none of its numbers were changed**.

A re-fit was necessary rather than a re-label because the 2026-10-28 run never stored its predictions. The re-fit is deterministic on this machine (two runs bit-identical) but is not the same fit: it reproduces Linear and Transformer Top-1 to within `0.01`, while the two threshold-dependent arms move by `0.027`. That drift concentrates exactly at the `P_trf(4) \ge 0.35` boundary, which is itself the evidence for the reframing in §4.3 — a hard cutoff on one class posterior is numerically brittle as well as mistuned. The recovery arms (`46/331`, `23/331`, `73/331`), the paired tests, and the systems block were **not** re-run and are unchanged.

2026-10-29 — §1.3 described a control that does not exist. The section reported `+0.105` "vanishing when matched against parameter-equivalent LoRA adapters (`8.15M` vs `442K`)". Three problems: `8.15M` vs `442K` is not parameter-equivalent — it is the *unmatched* comparison, and it produced `+0.135`, not `+0.105`; the matched control is `22.3M` vs `22.3M` and produced `+0.035 [-0.045, +0.115]`, not significant; and `+0.105` does not appear in any artifact in `results/`. §1.3 now reports the matched control from `findings/2026-10-25-steering-capacity-control.md`, and the unmatched `+0.135` is named as what it was.

These corrections change conclusions rather than wording, which is why they are listed here instead of being edited away.

Reproducibility & Source Verification

The following code and recorded artifacts are available locally or referenced by the repository. This is not a claim that every historical prediction or checkpoint was saved, that every local file is versioned, or that fresh-clone regeneration is exact. Original localizer predictions are missing; re-fits differ; model assets and dependencies remain external. See `REPRODUCIBILITY.md` and the evidence audit:

- **Harness & Pipelines:** `experiments/path_a_loop.py`, `experiments/eval_vault.py`, `experiments/interp_calls.py`
- **Raw JSON Metrics:** `results/path_a_loop_h5.json`, `results/verify_vault_h5.json`, `results/verify_vault_h5_localizer_refit.json`, `results/verify_h10_scaling.json`, `results/oracle_rewind_h5.json`

- **Underlying Corpus:** `results/arith_steps_h5.pt` (tuning), `results/arith_steps_h5_vault.pt` (blind vault)
- **Detailed Finding Reports:** `findings/2026-10-28-out-of-sample-and-h10-verification.md` (historical vault + H=10 report; systems interpretation superseded by the 2026-10-30 audit); `findings/2026-10-28-path-a-ensemble-kv-slicing.md` (Path A; corrected in place on 2026-10-28 after the vault audit); `findings/2026-10-25-steering-capacity-control.md` (capacity control cited in §1.3)